

ÍNDICE

1. INTRODUÇÃO.....	2
2. ARQUITETURA DO SISTEMA.....	3
2.1. BANCO DE DADOS.....	3
2.2. BACKEND.....	5
2.2.1. Extração dos dados.....	5
2.2.2. Transformação dos dados.....	7
2.2.3. Busca dos e-mails.....	8
2.2.4. Enriquecimento por Inteligência Artificial.....	9
2.2.5. Execução modular.....	11
2.3. API.....	12
2.3.1. Implementação.....	12
2.3.2. Execução.....	12
2.3.3. Endpoints.....	12
2.4. FRONTEND.....	14
2.4.1. Implementação.....	14
2.4.2. Interface.....	14
2.4.3. Comunicação com a API.....	17
2.4.4. Execução.....	17
3. CONCLUSÃO.....	18
REFERÊNCIAS.....	19

1. INTRODUÇÃO

As universidades produzem continuamente trabalhos acadêmicos que representam importantes contribuições para o avanço científico e tecnológico. Entretanto, grande parte desse conhecimento permanece restrito ao meio acadêmico, uma vez que os repositórios institucionais são voltados principalmente ao armazenamento e à preservação das publicações, apresentando seus conteúdos de forma técnica e pouco acessível ao público em geral.

Nesse contexto, a divulgação científica desempenha um papel fundamental ao aproximar a sociedade das pesquisas desenvolvidas nas universidades. Tornar essas informações mais compreensíveis contribui para ampliar a visibilidade da produção científica, facilitar o acesso ao conhecimento e evidenciar o impacto das instituições de ensino e pesquisa na sociedade.

Este projeto teve como objetivo desenvolver um sistema capaz de automatizar a obtenção de trabalhos acadêmicos disponibilizados pelo repositório institucional da Universidade Estadual Paulista (UNESP), processar seus metadados, enriquecer suas informações com o auxílio de inteligência artificial e disponibilizar os resultados em uma interface web voltada ao público geral.

Para isso, foi implementado um pipeline de processamento responsável pela extração dos dados da API pública da UNESP, transformação e normalização dos metadados, obtenção dos e-mails institucionais de autores e orientadores e geração de conteúdos complementares por meio de um modelo de linguagem. As informações processadas são armazenadas em um banco de dados local e disponibilizadas por meio de uma API REST consumida pelo frontend da aplicação.

Os capítulos seguintes apresentam a arquitetura do sistema, descrevendo os componentes desenvolvidos e o funcionamento de cada etapa do processamento dos dados.

O código-fonte desenvolvido durante o projeto encontra-se disponível publicamente no GitHub, permitindo sua consulta, utilização e continuidade por outros desenvolvedores. Disponível em: <https://github.com/felipedelfin00/unesp-paper-cards>.

2. ARQUITETURA DO SISTEMA

Este capítulo apresenta a arquitetura do sistema desenvolvido, descrevendo os principais componentes da aplicação e suas responsabilidades. São abordados o banco de dados, o backend responsável pelo processamento dos trabalhos acadêmicos, a API utilizada para disponibilização dos dados e o frontend, responsável pela interface com o usuário.

2.1. BANCO DE DADOS

Para o armazenamento das informações do sistema foi utilizado o SQLite, um sistema de gerenciamento de banco de dados relacional baseado em arquivos. A escolha foi motivada principalmente pela simplicidade de utilização durante o desenvolvimento do projeto.

Por ser um banco de dados embarcado, o SQLite não requer a instalação ou configuração de um servidor dedicado. Todo o banco é armazenado em um único arquivo, permitindo que a aplicação seja executada localmente de forma imediata, reduzindo a complexidade da configuração do ambiente de desenvolvimento.

O banco armazena todas as informações referentes aos trabalhos acadêmicos extraídos da API da UNESP, incluindo seus metadados e os conteúdos gerados durante o processamento da aplicação.

O arquivo do banco de dados está localizado em `backend/data/papers.db`. Os módulos responsáveis pela criação da estrutura do banco, definição dos modelos e gerenciamento das conexões encontram-se no diretório `backend/app/database`.

Caso o arquivo `papers.db` não exista, execute o comando `python main.py --database`. Esse procedimento cria automaticamente o banco de dados e todas as tabelas necessárias para o funcionamento da aplicação.

A estrutura do banco de dados é composta por uma tabela principal e as tabelas auxiliares responsáveis pelo armazenamento das informações complementares de cada trabalho acadêmico. A Tabela 1 apresenta a finalidade de cada uma delas.

TABELA	FINALIDADE
papers	Armazena as informações principais de cada trabalho acadêmico, como título, resumo, ano de emissão, idioma e demais metadados.
advisors	Armazena os orientadores dos trabalhos, seus e-mails institucionais e sua associação ao respectivo registro em papers.
authors	Armazena os autores dos trabalhos, seus e-mails institucionais e sua associação ao respectivo registro em papers.
campuses	Armazena os campi aos quais os trabalhos estão vinculados e sua associação ao respectivo registro em papers.
clicks	Armazena o registro de acessos recebidos por cada trabalho no website e sua associação ao respectivo registro em papers.
keywords	Armazena as palavras-chave de cada trabalho e sua associação ao respectivo registro em papers.
sdg	Armazena os Objetivos de Desenvolvimento Sustentável (ODS) associados a cada trabalho, a justificativa para essa associação e sua associação ao respectivo registro em papers.

Tabela 1: Finalidades das tabelas no banco de dados.

2.2. BACKEND

O backend do sistema foi desenvolvido em Python e é responsável pela obtenção, processamento e armazenamento das informações utilizadas pela aplicação. Sua implementação foi organizada de forma modular, permitindo que cada etapa do processamento seja executada de maneira independente. Essa abordagem facilita a manutenção do código, a identificação de falhas e a reexecução de etapas específicas do fluxo de processamento sempre que necessário.

2.2.1. Extração dos dados

A etapa de extração é responsável por obter os trabalhos acadêmicos disponibilizados pela API pública do repositório da UNESP. Antes de iniciar esse processo, o sistema consulta o banco de dados para identificar quais trabalhos já foram processados. Essa verificação é realizada por meio do UUID de cada registro, evitando a duplicação de dados e reduzindo a quantidade de requisições desnecessárias à API.

Em seguida, o sistema consulta a API do repositório da UNESP, disponível em <https://repositorio.unesp.br/server/api>, utilizando um filtro para retornar apenas os trabalhos pertencentes ao Instituto de Geociências e Ciências Exatas (IGCE) e ao Instituto de Biociências (IB). Para cada item retornado, o UUID é comparado com os registros já existentes no banco de dados. Caso o trabalho ainda não tenha sido processado, seu identificador é adicionado à lista de extração da execução atual.

Após a definição dessa lista, o sistema realiza uma nova consulta à API para cada UUID selecionado, obtendo todos os metadados disponibilizados pelo repositório. Os dados extraídos são armazenados temporariamente em um arquivo no formato JSON Lines (JSONL), que será utilizado para a etapa de transformação. Esse arquivo é armazenado em [backend/data/raw.jsonl](#).

A utilização de um arquivo intermediário foi adotada porque os dados retornados pela API ainda não estão adequados ao modelo do banco de dados. Dessa forma, a etapa de transformação pode realizar toda a limpeza, normalização e reorganização das informações antes de sua inserção definitiva no SQLite.

Para controlar o volume de requisições realizadas à API e evitar o armazenamento de grandes quantidades de dados ainda não processados, o módulo de extração possui um limite configurável de trabalhos por execução. Esse limite é definido no arquivo main.py, e é aplicado individualmente para cada campus. Assim, ao definir um limite de 50 registros, por exemplo, serão extraídos até 50 trabalhos do IGCE e até 50 trabalhos do IB.

Essa limitação também foi adotada devido ao fluxo de processamento do sistema. A etapa de enriquecimento por inteligência artificial representa o principal gargalo da aplicação, uma vez que está sujeita a limites de utilização da API do modelo de linguagem. Dessa forma, extrair apenas a quantidade de trabalhos que poderá ser efetivamente processada evita o acúmulo de dados que ainda não serão disponibilizados no website, tornando a execução mais eficiente.

2.2.2. Transformação dos dados

A etapa de transformação é responsável por converter os dados extraídos da API da UNESP para o formato utilizado pelo sistema. Nessa fase, os metadados são selecionados, normalizados e organizados de acordo com a estrutura do banco de dados, tornando-os adequados para as etapas seguintes do processamento.

Inicialmente, o módulo realiza a leitura do arquivo JSONL gerado durante a etapa de extração. Para cada trabalho acadêmico presente no arquivo, são selecionados apenas os metadados necessários ao funcionamento do sistema, descartando informações que não serão utilizadas.

Após essa seleção, cada campo passa pelo processo de normalização correspondente. Dependendo do tipo de dado, podem ser realizadas operações como padronização de textos, remoção de caracteres desnecessários, conversão de formatos de dados e tratamento de listas retornadas pela API. Essas rotinas estão centralizadas no arquivo [backend/app/utils/normalize.py](#), facilitando sua manutenção e reutilização.

Concluída a transformação de todos os registros, os dados são persistidos no banco de dados SQLite, respeitando a estrutura das tabelas da aplicação. Ao término do processo, o arquivo JSONL é removido automaticamente, uma vez que sua função é apenas servir como armazenamento temporário entre as etapas de extração e transformação.

2.2.3. Busca dos e-mails

A etapa de busca dos e-mails institucionais tem como objetivo complementar os dados dos autores e orientadores armazenados no banco de dados, permitindo que o website disponibilize os contatos institucionais dos pesquisadores.

Como não foi encontrada uma API pública que fornecesse essas informações, optou-se pela utilização de uma rotina de web scraping sobre o Portal de Docentes da UNESP. Dessa forma, foi possível automatizar a obtenção dos e-mails institucionais, sem a necessidade de consultas manuais.

Inicialmente, o sistema consulta o banco de dados para identificar todos os autores e orientadores cujo e-mail institucional ainda não foi obtido. Em seguida, para cada registro, é realizada uma pesquisa no Portal de Docentes da UNESP (<https://unesp.br/portaldocentes/>) utilizando o nome do pesquisador como critério de pesquisa.

Como o Portal de Docentes não disponibiliza uma API pública para consulta dos dados dos pesquisadores, o sistema analisa o conteúdo HTML da página de resultados para localizar o primeiro perfil correspondente. Em seguida, acessa a página individual desse docente e procura pelo elemento HTML identificado pelo atributo `id="email-docente"`. Quando esse elemento está presente, seu conteúdo é extraído e armazenado no banco de dados.

Caso nenhum perfil seja encontrado ou o portal não disponibilize um e-mail institucional para o pesquisador, o campo correspondente permanece com valor vazio e o campo `verified` é definido como `yes`, indicando que a busca já foi realizada.

O algoritmo considera apenas o primeiro resultado retornado pelo Portal de Docentes, assumindo que ele corresponde ao pesquisador buscado. Embora essa abordagem tenha apresentado bons resultados durante o desenvolvimento, ela pode gerar associações incorretas em casos de nomes comuns ou homônimos.

2.2.4. Enriquecimento por Inteligência Artificial

A etapa de enriquecimento por inteligência artificial é a última do pipeline de processamento. Ao final dessa etapa, os trabalhos acadêmicos possuem todas as informações necessárias para serem apresentados no website. Seu principal objetivo é transformar informações técnicas em conteúdos acessíveis ao público geral, despertando o interesse pela pesquisa e facilitando a compreensão dos trabalhos desenvolvidos na universidade.

O modelo utilizado é o OpenAI GPT OSS 120B, disponibilizado pela plataforma Groq. A escolha desse modelo foi motivada pelo bom desempenho observado durante o desenvolvimento do projeto, pela disponibilidade de uma cota gratuita de utilização e pela capacidade de processamento oferecida pela plataforma, suficiente para atender à demanda da aplicação.

Para que o módulo funcione corretamente, é necessário configurar uma chave de acesso da plataforma Groq no arquivo `backend/app/.env`. Como essa chave é pessoal, ela não foi incluída no projeto e deverá ser obtida pelo responsável pela implantação do sistema.

Durante o enriquecimento, o sistema envia ao modelo um prompt contendo uma persona de jornalista especializado em divulgação científica, instruções sobre o formato esperado da resposta, regras para a geração dos conteúdos, o título do trabalho e seu resumo. Para reduzir o consumo de tokens, o resumo é truncado para aproximadamente 2000 caracteres antes do envio ao modelo.

Como resultado, a inteligência artificial gera um conjunto de informações adicionais utilizadas pelo website, incluindo:

- Um resumo em linguagem acessível;
- A relevância do trabalho para a sociedade;
- A área do conhecimento correspondente;
- Os Objetivos de Desenvolvimento Sustentável (ODS) relacionados ao trabalho, acompanhados de suas respectivas justificativas;
- Um tema para classificação do trabalho, escolhido entre uma lista pré-definida pela aplicação e utilizado na aparência do card no frontend.

O prompt utilizado nessa etapa está implementado no arquivo [backend/app/services/enrich/prompt.py](#), permitindo que as instruções fornecidas ao modelo sejam modificadas sem necessidade de alterar o restante da lógica de comunicação com a API.

Assim como na etapa de extração, o módulo de enriquecimento possui um limite configurável de trabalhos processados por execução, definido no arquivo [main.py](#). Esse limite foi estabelecido considerando a capacidade diária de processamento do modelo utilizado, evitando que o sistema interrompa sua execução devido ao esgotamento da cota disponível.

Além desse limite preventivo, o sistema implementa um mecanismo de segurança para tratar erros relacionados ao limite de utilização da API. Caso a API retorne uma resposta indicando que a cota diária foi atingida, o processamento é interrompido imediatamente, preservando os dados já enriquecidos e permitindo que a execução seja retomada posteriormente.

2.2.5. Execução modular

O backend foi desenvolvido seguindo uma arquitetura modular, na qual cada etapa do pipeline de processamento pode ser executada de forma independente. Essa abordagem facilita a manutenção da aplicação, a identificação de falhas e a execução parcial do fluxo, evitando o reprocessamento de etapas que já foram concluídas.

A execução dos módulos é realizada por meio de argumentos de linha de comando processados pelo arquivo main.py. Cada argumento corresponde a uma etapa específica do pipeline, permitindo que apenas a funcionalidade desejada seja executada. Também é disponibilizado um modo de execução completa, responsável por executar todas as etapas em sequência.

O arquivo main.py atua como ponto de entrada da aplicação, interpretando os argumentos de linha de comando e iniciando o módulo correspondente. Cada etapa do pipeline é implementada em um módulo localizado no diretório backend/app/jobs, responsável por coordenar sua execução. A lógica de processamento de cada etapa encontra-se no diretório backend/app/services, onde estão implementadas as funcionalidades utilizadas pelos respectivos módulos.

A Tabela 2 apresenta os argumentos disponíveis para execução do backend.

ARGUMENTO	DESCRIÇÃO
--database	Cria a estrutura do banco de dados SQLite, caso ainda não exista.
--extract	Executa apenas a etapa de extração dos dados da API da UNESP.
--transform	Executa apenas a etapa de transformação dos dados extraídos.
--contacts	Executa a busca dos e-mails institucionais dos autores e orientadores.
--enrich	Executa o enriquecimento dos trabalhos por meio da IA.
--process	Executa todas as etapas do pipeline de processamento em sequência.

Tabela 2: Argumentos disponíveis para execução.

2.3. API

A API é responsável por disponibilizar ao frontend os dados armazenados no banco de dados. Ela atua como intermediária entre interface do usuário e o backend, centralizando o acesso às informações e evitando que o website se comunique diretamente com o banco de dados.

A separação entre o pipeline de processamento e a API permite que a extração e o enriquecimento dos dados ocorram independentemente da interface do usuário. Dessa forma, o website consome apenas dados já processados, reduzindo o tempo de resposta e evitando consultas diretas às fontes externas durante a navegação.

2.3.1. Implementação

A API foi desenvolvida em Python utilizando o framework FastAPI e é executada localmente por meio do servidor Uvicorn. Essas tecnologias foram escolhidas por oferecerem uma implementação mais simples, boa integração com o restante da aplicação e suporte à criação de APIs REST de forma eficiente.

Os endpoints da API estão implementados no arquivo `backend/app/api/routes/cards.py`. A lógica responsável pelas consultas ao banco de dados foi separada em módulos localizados no diretório `backend/app/api/repositories`, facilitando a manutenção e reutilização do código.

2.3.2. Execução

A API é executada localmente utilizando o servidor Uvicorn e pode ser iniciada por meio do comando `uvicorn api.routes.cards:app`. Como alternativa, o script `run.sh` inicializa automaticamente tanto a API quanto o servidor responsável pelo frontend, simplificando a execução da aplicação durante o desenvolvimento.

2.3.3. Endpoints

A API disponibiliza apenas os dados necessários para o funcionamento do website, mantendo separadas as rotinas de processamento e de apresentação. Dessa forma, as requisições realizadas pelo frontend são respondidas exclusivamente

a partir dos dados armazenados no banco de dados, sem necessidade de novas consultas à API da UNESP ou ao modelo de inteligência artificial. A Tabela 3 apresenta os endpoints da API e suas finalidades.

MÉTODO	ROTA	FINALIDADE
GET	/cards	Retorna a lista de trabalhos, podendo receber parâmetros de busca e filtragem.
GET	/cards/{pid}	Retorna os dados de um trabalho específico
POST	/cards/{pid} /click	Registra um acesso ao trabalho.
GET	/filters	Retorna os valores disponíveis para os filtros utilizados pelo frontend.

Tabela 3: Endpoints da API.

2.4. FRONTEND

O frontend é responsável pela apresentação das informações processadas pelo backend, permitindo que os usuários consultem e explorem os trabalhos acadêmicos por meio de uma interface web. Todas as informações exibidas são obtidas por meio da API da aplicação, não havendo comunicação direta com o banco de dados nem com o pipeline de processamento.

2.4.1. Implementação

O frontend foi desenvolvido utilizando HTML, CSS e JavaScript, sem o uso de bibliotecas ou frameworks externos. Essa escolha foi motivada pela simplicidade da interface da aplicação, reduzindo a quantidade de dependências do projeto e facilitando sua manutenção.

Os arquivos do frontend encontram-se no diretório frontend, organizados de acordo com sua finalidade.

2.4.2. Interface

A aplicação é composta por uma única página, responsável por apresentar os trabalhos acadêmicos em formato de cards. Ao acessar o website, o frontend consulta a API e exibe uma versão resumida de cada trabalho, contendo informações como título, resumo, área do conhecimento, tipo do trabalho, idioma e ano de emissão.

Além da listagem dos trabalhos, a interface disponibiliza mecanismos de pesquisa e filtragem. Os filtros textuais permitem que o usuário pesquise diretamente por termos de interesse, enquanto os filtros de seleção obtêm dinamicamente seus valores por meio da API, garantindo que as opções apresentadas estejam sempre sincronizadas com os dados disponíveis no banco de dados.

Ao selecionar um card, o usuário pode visualizar informações adicionais sobre o trabalho acadêmico, incluindo os dados gerados durante a etapa de enriquecimento por inteligência artificial e outras informações complementares armazenadas pela aplicação.

Para melhorar a experiência de navegação, os trabalhos são carregados de forma incremental por meio de uma estratégia de rolagem infinita. Novos registros são requisitados automaticamente quando o usuário se aproxima do final da página, reduzindo o tempo de carregamento inicial e melhorando a navegação em listas extensas.

Além das informações textuais, cada card apresenta elementos visuais definidos a partir dos dados gerados pela inteligência artificial. O tema atribuído ao trabalho determina sua cor e seu ícone representativo, facilitando a identificação visual dos diferentes assuntos abordados.

A interface foi desenvolvida para se adaptar a diferentes tamanhos de tela, permitindo sua utilização tanto em computadores quanto em dispositivos móveis.

As Imagens 1, 2 e 3 apresentam a interface do website.

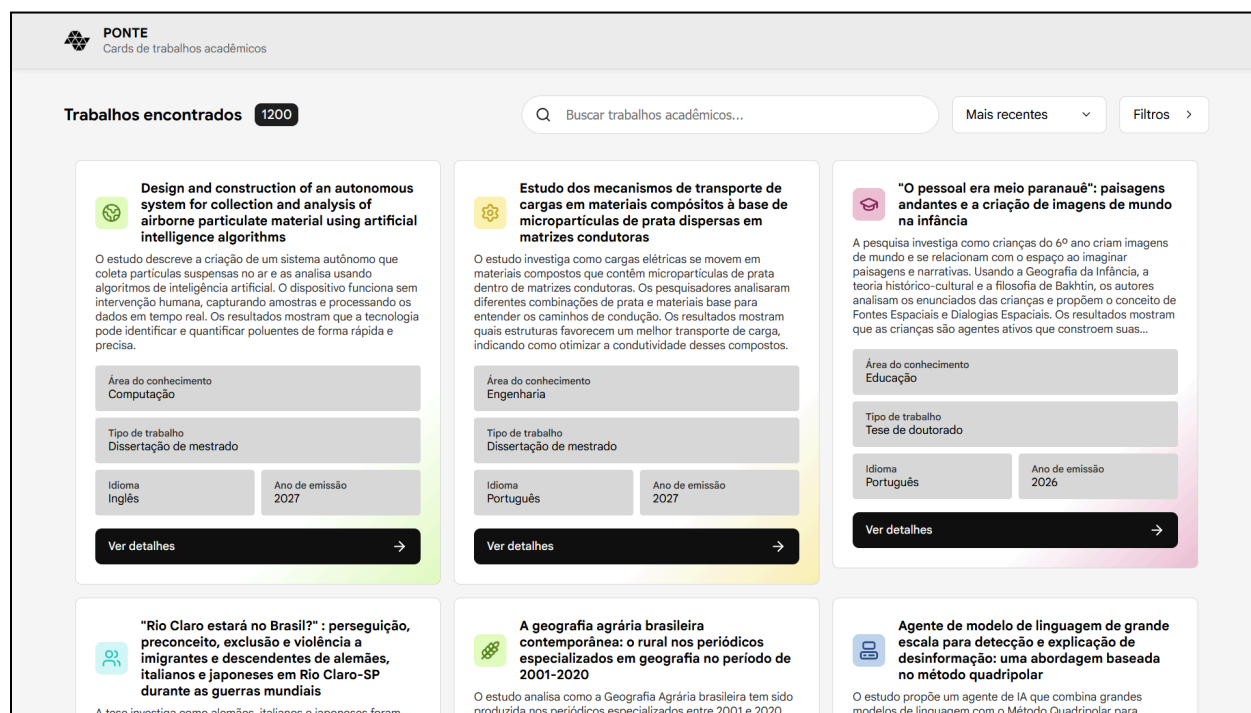


Figura 1: Página inicial do website.

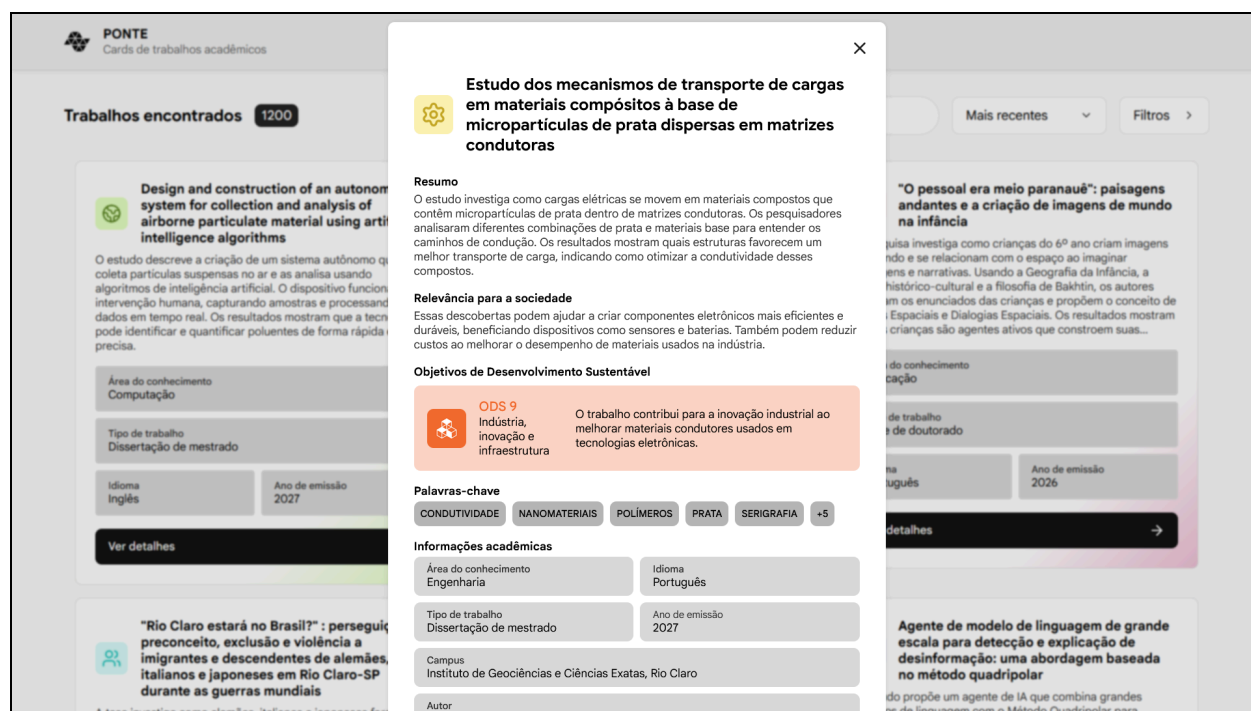


Figura 2: modal com as informações completas do card.

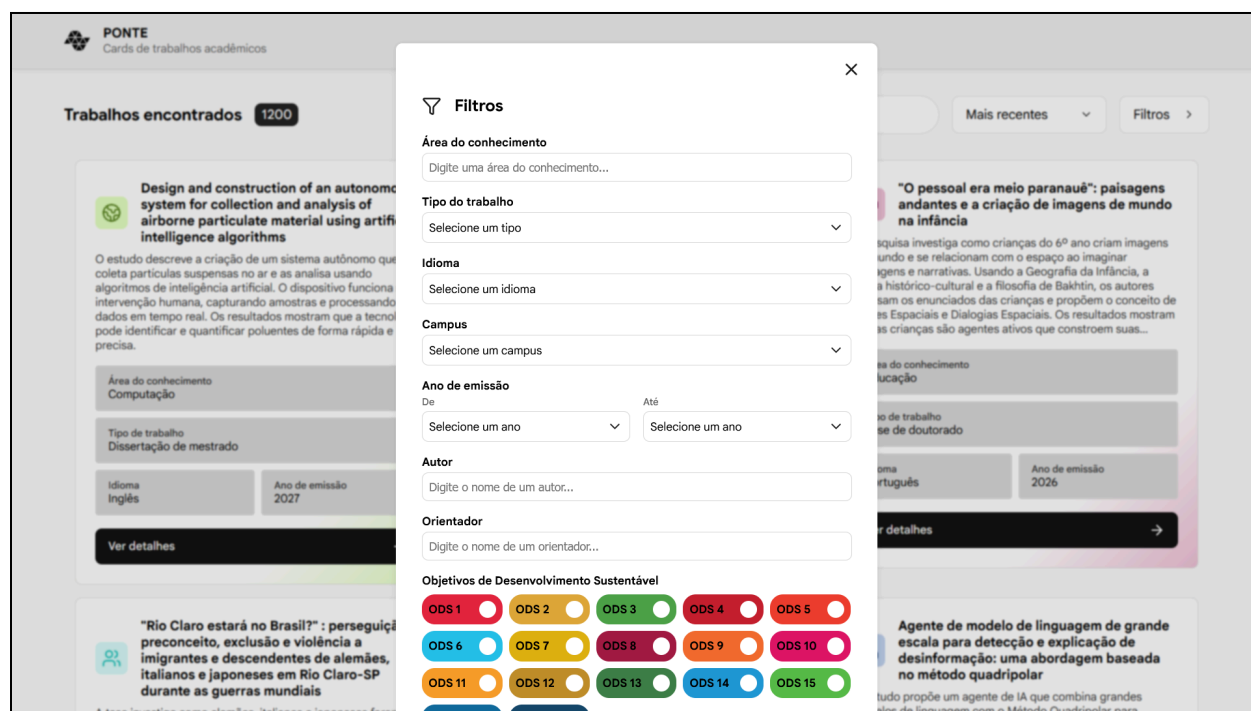


Figura 3: modal com os filtros aplicáveis.

2.4.3. Comunicação com a API

O frontend realiza todas as consultas por meio da API da aplicação. A listagem dos trabalhos e a aplicação de filtros utilizam o endpoint GET /cards, enquanto a visualização completa de um trabalho utiliza o endpoint GET /cards/{pid}.

Quando o usuário acessa o link do trabalho original, o frontend envia uma requisição ao endpoint POST /cards/{pid}/click, permitindo o registro da quantidade de acessos recebidos por cada publicação. Como mecanismo de controle, o sistema contabiliza apenas um clique por usuário para um mesmo trabalho durante um período de uma hora, evitando a superestimação do número de acessos.

2.4.4. Execução

O frontend é executado localmente utilizando o servidor HTTP disponibilizado pelo Python e pode ser iniciado por meio do comando python -m http.server 5500. Como alternativa, o script run.sh inicializa automaticamente tanto o frontend quanto o servidor responsável pela API, simplificando a execução da aplicação durante o desenvolvimento.

3. CONCLUSÃO

Este projeto resultou no desenvolvimento de um sistema capaz de automatizar a obtenção, o processamento e a divulgação de trabalhos acadêmicos da Universidade Estadual Paulista (UNESP). A solução integra a extração de dados do repositório institucional, a transformação e normalização dos metadados, a obtenção dos conteúdos por meio de inteligência artificial, disponibilizando os resultados em uma interface web voltada ao público geral.

Os objetivos propostos foram alcançados, permitindo a construção de uma aplicação capaz de transformar informações originalmente técnicas em conteúdos mais acessíveis e organizados. Dessa forma, o projeto contribui diretamente para o propósito de divulgação científica apresentado na introdução: aproximar a sociedade das pesquisas desenvolvidas na universidade, tornando visível uma produção acadêmica que, de outra forma, permaneceria restrita aos repositórios institucionais e ao público especializado.

A arquitetura modular adotada durante o desenvolvimento contribui para a manutenção e evolução do sistema, uma vez que cada etapa do pipeline pode ser executada e modificada de forma independente.

Ao longo do desenvolvimento, também foram identificadas oportunidades de aprimoramento para futuras versões do sistema. No processo de busca de e-mails institucionais, por exemplo, a estratégia de considerar o primeiro resultado retornado pelo Portal de Docentes pode ser refinada para reduzir associações incorretas em casos de nomes homônimos. Da mesma forma, o processo de enriquecimento por inteligência artificial poderia ser expandido para atender a um volume maior de trabalhos, à medida que novas cotas ou modelos alternativos se tornem disponíveis. Outras possibilidades de evolução incluem a incorporação de novas funcionalidades de pesquisa e visualização, a ampliação da cobertura para outros institutos da universidade e a implantação da aplicação em ambiente de produção, permitindo seu acesso pela comunidade acadêmica e pela sociedade em geral.

REFERÊNCIAS

DELFINO, Felipe Rovigatti. Cards de Trabalhos Acadêmicos da UNESP. GitHub. Disponível em: <https://github.com/felipedelfin00/unesp-paper-cards>. Acesso em: 16 jul. 2026.

FASTAPI. FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 16 jul. 2026.

GROQ. Groq Documentation. Disponível em: <https://console.groq.com/docs/overview>. Acesso em: 16 jul. 2026.

SQLITE. SQLite Documentation. Disponível em: <https://sqlite.org/docs.html>. Acesso em: 16 jul. 2026.

UNESP. API do Repositório Institucional. Disponível em: <https://repositorio.unesp.br/server/api>. Acesso em: 16 jul. 2026.

UNESP. Portal de Docentes. Disponível em: <https://unesp.br/portaldocentes>. Acesso em: 16 jul. 2026.